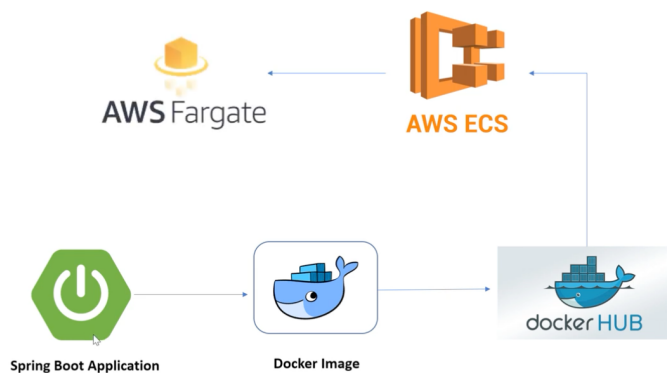
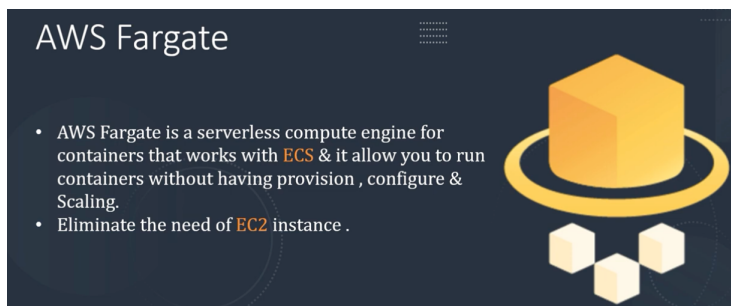
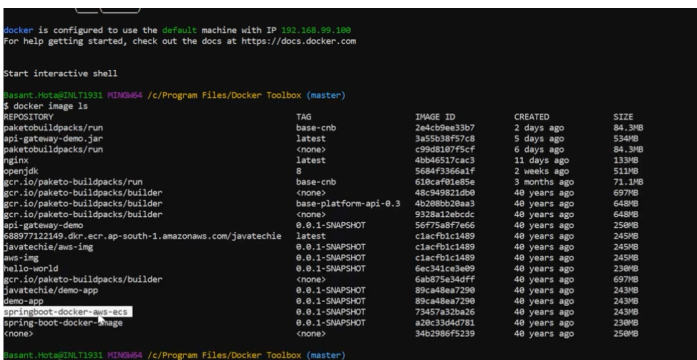
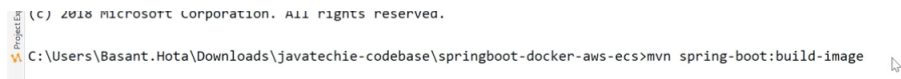
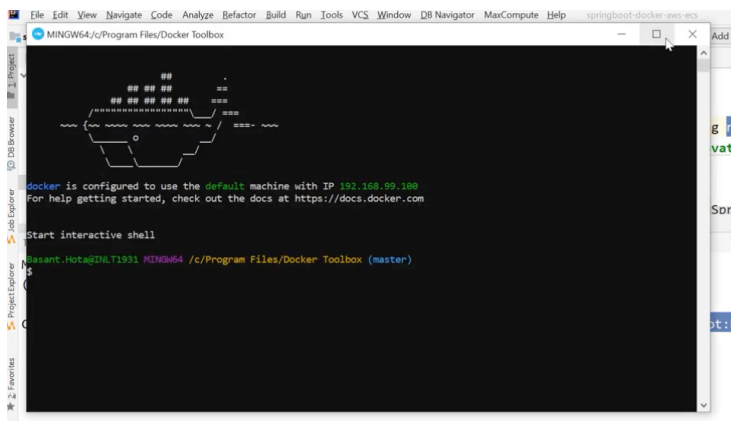
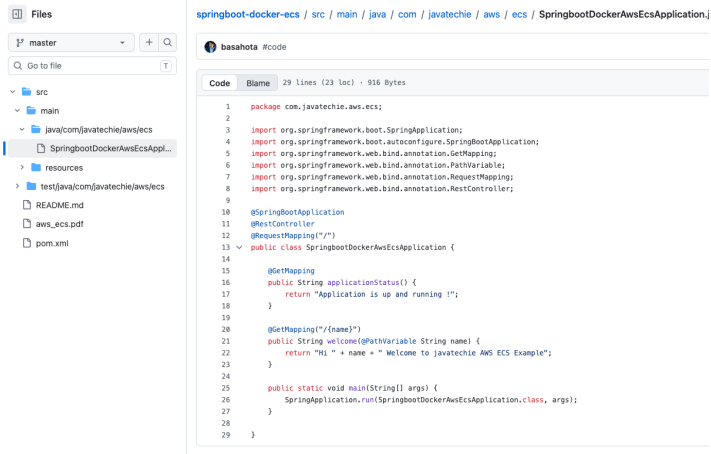


Spring Boot Application – Deploy Docker image from docker hub and AWS Elastic Container Registry to ECS

AWS ECS – Docker – scaling and maintenance is easy.



<https://github.com/Java-Techie-jt/springboot-docker-ecs>



Run the docker image

```
[INFO] -----
C:\Users\Basant.Hota\Downloads\javatechie-codebase\springboot-docker-aws-ecs>docker run --tty --publish 8080:8080 springboot-docker-aws-ecs:0.0.1-SNAPSHOT
```

User docker ip address instead of localhost

← → ↻ ⓘ Not secure | 192.168.99.100:8080/Basant

Hi Basant Welcome to javatechie AWS ECS Example

Create tag for docker image

```
Basant.Hota@INLT1931 MINGW64 /c/Program Files/Docker Toolbox (master)
$ docker tag springboot-docker-aws-ecs:0.0.1-SNAPSHOT javatechie/springboot-docker-aws-ecs:0.0.1-SNAPSHOT
```

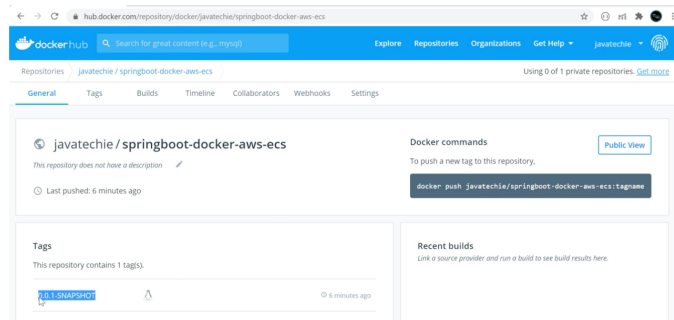
Now we will see two images

```
Basant.Hota@INLT1931 MINGW64 /c/Program Files/Docker Toolbox (master)
$ docker image ls
REPOSITORY                                TAG                IMAGE ID           CREATED           SIZE
paketobuildpacks/run                     base-cnb           2e4cb9ee33b7      2 days ago       84.3MB
api-gateway-demo.jar                     latest            3a55b38f57c8      5 days ago       534MB
paketobuildpacks/run                     <none>            c99d8107f5cf      6 days ago       84.3MB
nginx                                     latest            4bb46517cac3      11 days ago      133MB
openjdk                                  8                 5684f3366a1f      2 weeks ago      511MB
gcr.io/paketo-buildpacks/run              base-cnb           610caf01e85e      3 months ago     71.1MB
spring-boot-docker-image                  0.0.1-SNAPSHOT     a20c33d4d781      40 years ago     230MB
gcr.io/paketo-buildpacks/builder          base-platform-api-0.3 4b208bb20aa3      40 years ago     648MB
javatechie/demo-app                      0.0.1-SNAPSHOT     89ca48ea7290      40 years ago     243MB
demo-app                                0.0.1-SNAPSHOT     89ca48ea7290      40 years ago     243MB
gcr.io/paketo-buildpacks/builder          <none>             6ab875e34dff      40 years ago     697MB
javatechie/springboot-docker-aws-ecs      0.0.1-SNAPSHOT     73457a32ba26      40 years ago     243MB
springboot-docker-aws-ecs                 0.0.1-SNAPSHOT     73457a32ba26      40 years ago     243MB
688977122149.dkr.ecr.ap-south-1.amazonaws.com/javatechie latest            c1acfb1c1489      40 years ago     245MB
```

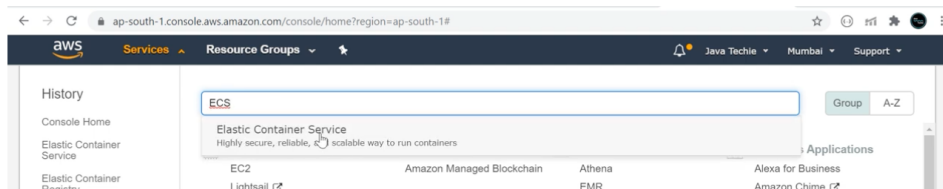
Push docker to docker hub. We can also push it to AWS ECR – Elastic container registry

```
Basant.Hota@INLT1931 MINGW64 /c/Program Files/Docker Toolbox (master)
$ docker push javatechie/springboot-docker-aws-ecs:0.0.1-SNAPSHOT
The push refers to repository [docker.io/javatechie/springboot-docker-aws-ecs]
8caf283d6182: Pushed
7cffc3c153c7: Mounted from javatechie/demo-app
f98424725587: Pushed
6763b09e3792: Mounted from javatechie/demo-app
5cd38b221a5e: Mounted from javatechie/demo-app
4f53326d9ef5: Mounted from javatechie/demo-app
810e950c6a17: Mounted from javatechie/demo-app
f5192aaff1a0: Mounted from javatechie/demo-app
6d0665b308e3: Mounted from javatechie/demo-app
1708488e8eef: Mounted from javatechie/demo-app
d520d6c22907: Mounted from javatechie/demo-app
e358996a2ee: Mounted from javatechie/demo-app
e84232dd4559: Mounted from javatechie/demo-app
079718509333: Mounted from javatechie/demo-app
7292be7bbc48: Mounted from javatechie/demo-app
2542fe3ce207: Mounted from javatechie/demo-app
6f5c8887c41a: Mounted from javatechie/demo-app
001e4a89773b: Mounted from javatechie/demo-app
2ba5b91ca2b0: Mounted from javatechie/demo-app
2f37d1102187: Mounted from javatechie/demo-app
79bde4d54386: Mounted from javatechie/demo-app
0.0.1-SNAPSHOT: digest: sha256:f27434b1deb7742e5a94c7bb951e2a7d3396e785d7f78a443bbe562cf486b56 size: 4718
Basant.Hota@INLT1931 MINGW64 /c/Program Files/Docker Toolbox (master)
$
```

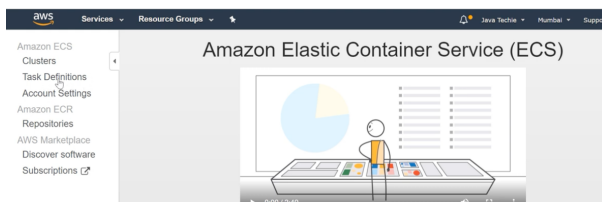
Now docker image is available in docker hub.



Now deploy it in ECS

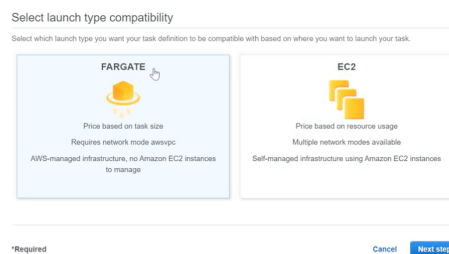


Create task definition



Create new Task Definition

Step 1: Select launch type compatibility
Step 2: Configure task and container definitions



Create new Task Definition

Step 1: Select launch type compatibility

Step 2: Configure task and container definitions

Configure task and container definitions

A task definition specifies which containers are included in your task and how they interact with each other. You can also specify data volumes for your containers to use. [Learn more](#)

Task Definition Name*.springboot-docker-ecs-example

Requires Compatibilities*FARGATE

Task RoleNone
Optional IAM role that tasks can use to make API requests to authorized AWS services. Create an Amazon Elastic Container Service Task Role in the IAM Console.

Task size

The task size allows you to specify a fixed size for your task. Task size is required for tasks using the Fargate launch type and is optional for the EC2 launch type. Container level memory settings are optional when task size is set. Task size is not supported for Windows containers.

Task memory (GB)

1GB

The valid memory range for 0.5 vCPU is: 1GB - 4GB.

Task CPU (vCPU)

0.5 vCPU

The valid CPU range for 1GB memory is: 0.25 vCPU - 0.5 vCPU.

Task memory maximum allocation for container memory reservation

0

1024 shared of 1024 MiB

Task CPU maximum allocation for containers

0

512 shared of 512 CPU units

Container Definitions

Add container

Container Nam...	Image	Hard/Soft mem...	CPU Unit...	GPU	Essential ...
No results					

Add container

Standard

Container name*

javatechie-docker-image

Image*

docker.io/javatechie/springboot-docker-aws-ecs:0.0.1-SNAPSHOT

Private repository authentication*

Memory Limits (MiB)

Hard limit

1024

Add Soft limit

Define hard and/or soft memory limits in MiB for your container. Hard and soft limits correspond to the 'memory' and 'memoryreservation' parameters, respectively, in task definitions.

ECS recommends 300-500 MiB as a starting point for web applications.

* Required

Cancel

Add

Port mappings

Container port	Protocol
8080	tcp
80	tcp

Add port mapping

Click on add and create buttons

Launch Status

Task definition status - 2 of 2 completed

Create Task Definition:.springboot-docker-ecs-example

springboot-docker-ecs-example succeeded

Create CloudWatch Log Group

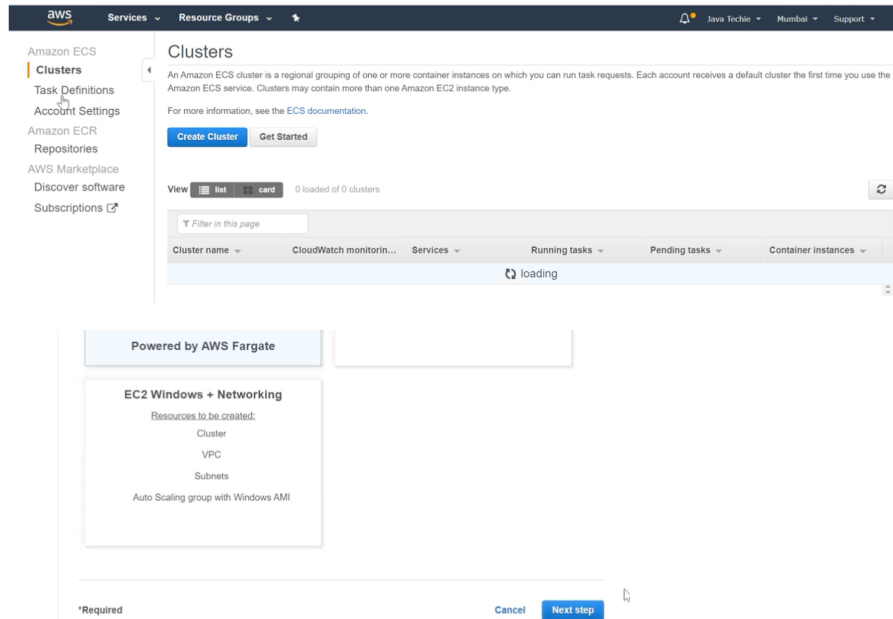
CloudWatch Log Group created

CloudWatch Log Group: /aws/springboot-docker-ecs-example

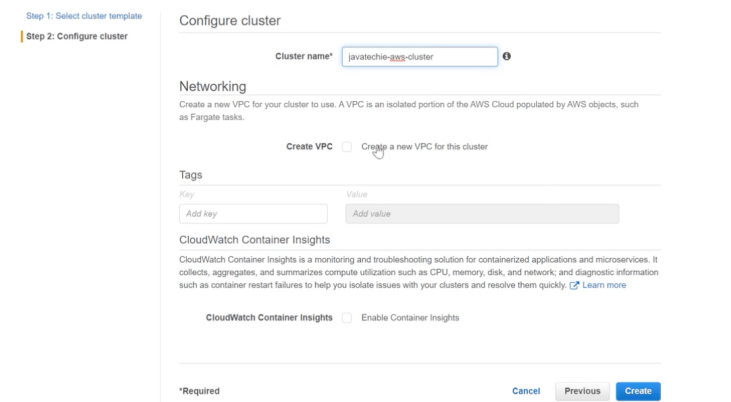
Back

View task definition

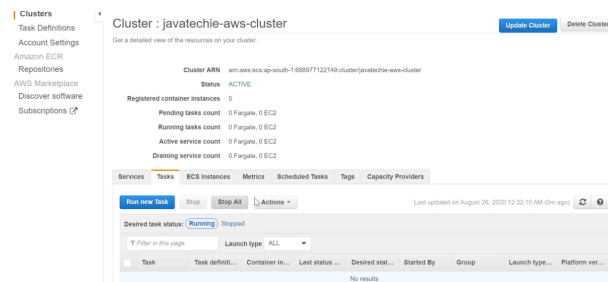
Then create cluster



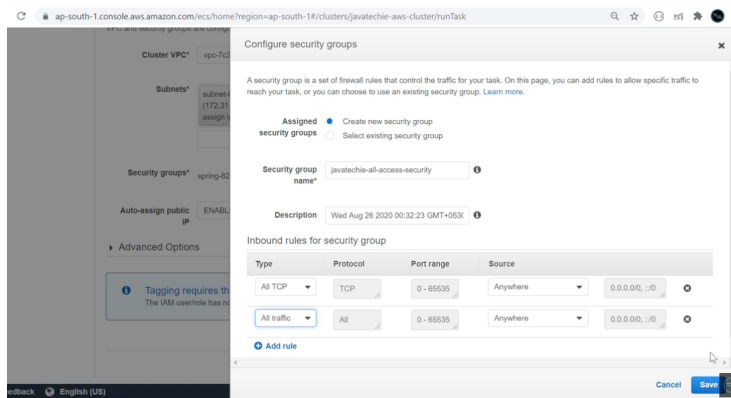
Here techie not selected create a new VPC for this cluster



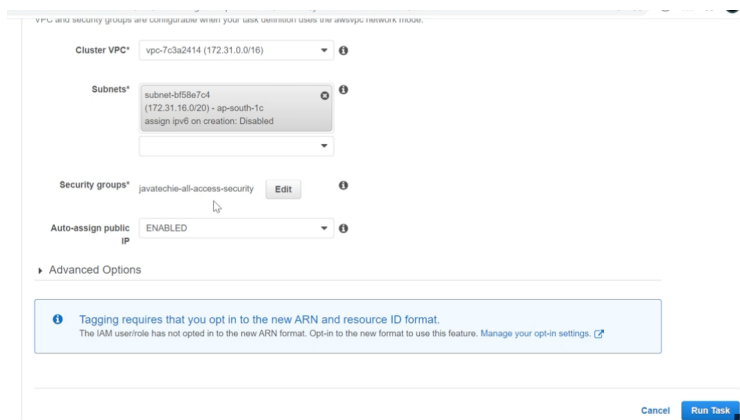
In this cluster we do not have any task.



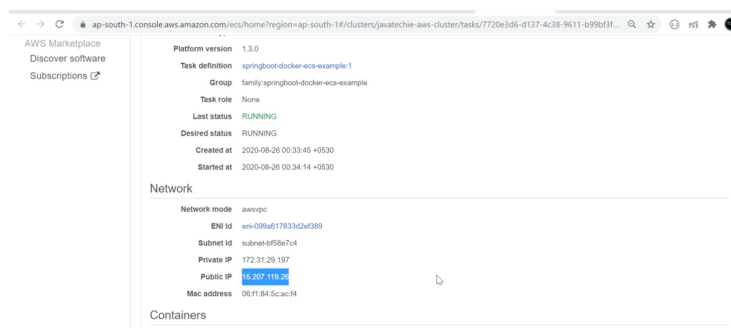
Add previously created task



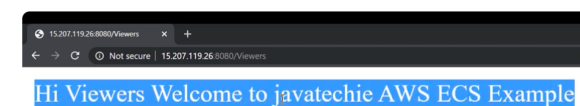
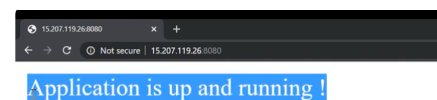
Select defaults and updated security groups.



Click run task



Take public IP and play around



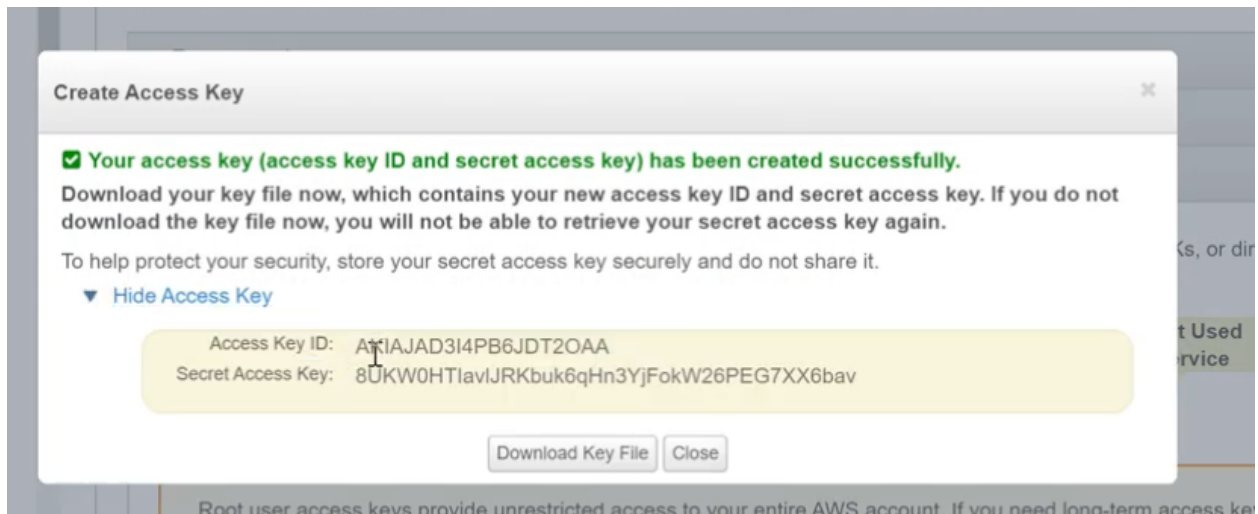
Now let us try with AWS ECR which is private hub instead of docker hub (which is public hub)



https://github.com/Java-Techie-jt/springboot-docker-ecs/blob/master/aws_ecs.pdf

The screenshot shows the AWS Management Console interface. The top navigation bar includes the AWS logo, 'Services', 'Resource Groups', and a user profile for 'Java Techie' in the 'Mumbai' region. The main content area is titled 'AWS Management Console'. On the left, there is a sidebar for 'Identity and Access Management (IAM)' with a 'Dashboard' and sections for 'Access management' (Groups, Users, Roles, Policies, Identity providers, Account settings) and 'Access reports' (Access analyzer, Archive rules). The main panel displays 'Your Security Credentials'. It includes instructions on how to manage credentials and a table for access keys. A 'Create New Access Key' button is visible at the bottom of the table.

Created	Access Key ID	Last Used	Last Used Region	Last Used Service	Status	Actions
Create New Access Key						



Now install AWS CLI

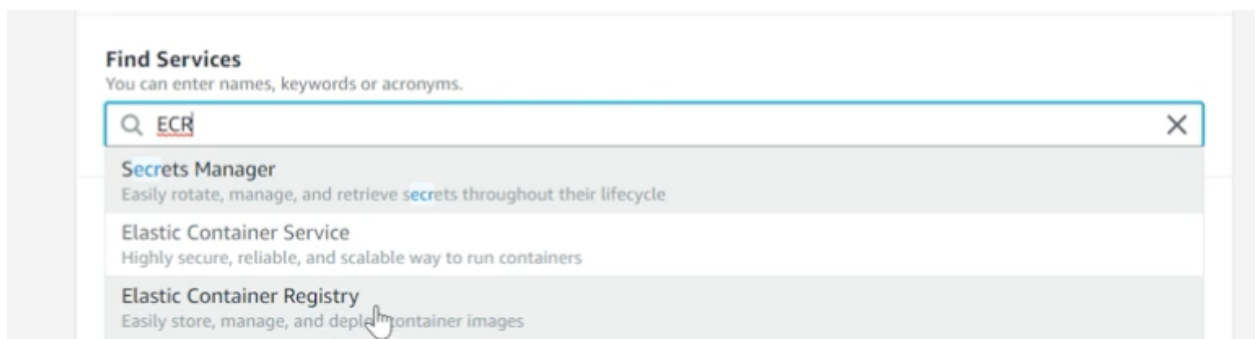
Install or update the AWS CLI version 2 on Windows using the MSI installer

1. Download the AWS CLI MSI installer for Windows (64-bit) at <https://awscli.amazonaws.com/AWSCLIV2.msi>
To update your current installation of AWS CLI version 2 on Windows, download a new installer each time you update to overwrite

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.17763.914]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Basant.Hota>aws configure
AWS Access Key ID [*****HQAIA]: AKIAJAD3I4PB6JDT2OAA
AWS Secret Access Key [*****u+dH]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json

C:\Users\Basant.Hota>
```



ECR > Repositories > Create repository

Create repository

Repository access and tags

Repository name

688977122149.dkr.ecr.ap-south-1.amazonaws.com/

A namespace can be included with your repository name (e.g. namespace/repo-name).

Tag immutability
Enable tag immutability to prevent image tags from being overwritten by subsequent image pushes using the same tag. Disable tag immutability to allow image tags to be overwritten.

☐ Disabled

Create repo

The screenshot shows the AWS ECR console with a green banner indicating 'Successfully created repository javatechie-registry'. Below this, a table lists the repository details:

Repository name	URI	Created at	Tag immutability	Scan on push	Encryption type
javatechie-registry	688977122149.dkr.ecr.ap-south-1.amazonaws.com/javatechie-registry	08/26/20, 11:54:11 PM	Disabled	Disabled	AES-256

Below the console, a terminal window shows the following commands:

```

1 AKIAJAD3I4PB6JDT2OAA
2
3 8UKW0HTIav1JRKbuk6qHn3YjFokW26PEG7XX6bav
4
5 688977122149.dkr.ecr.ap-south-1.amazonaws.com/javatechie-registry

```

```

C:\Users\Basant.Hota>aws configure
AWS Access Key ID [*****HQIA]: AKIAJAD3I4PB6JDT2OAA
AWS Secret Access Key [*****u+dh]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json

C:\Users\Basant.Hota>aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-st
din

```

```

C:\Users\Basant.Hota>aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-st
din 688977122149.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in C:\Users\Basant.Hota\.docker\config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded

C:\Users\Basant.Hota>

```

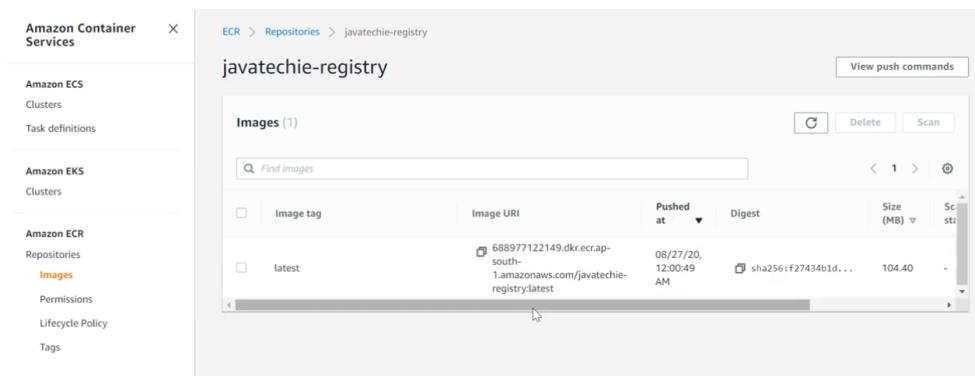
Create tag using previously created docker image

```
C:\Users\Basant.Hota>docker tag springboot-docker-aws-ecs:0.0.1-SNAPSHOT 688977122149.dkr.ecr.ap-south-1.amazonaws.com/javatechie-registry
C:\Users\Basant.Hota>
```

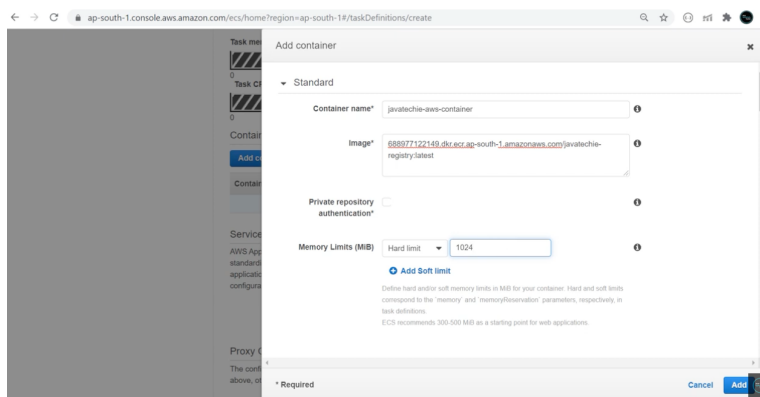
Push the image

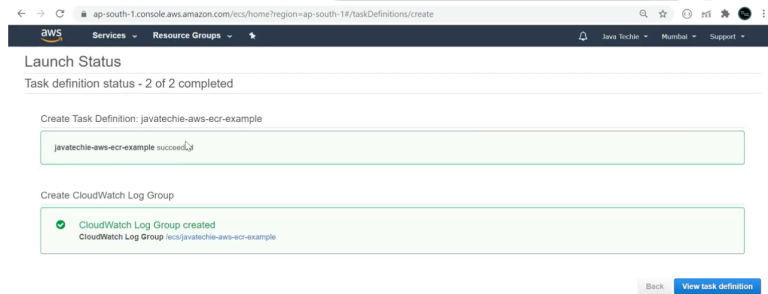
```
C:\Users\Basant.Hota>docker push 688977122149.dkr.ecr.ap-south-1.amazonaws.com/javatechie-registry
The push refers to repository [688977122149.dkr.ecr.ap-south-1.amazonaws.com/javatechie-registry]
8caf283d6182: Pushed
7cfc3c153c7: Pushed
f98424725587: Pushed
6763b99e3792: Pushed
5cd38b221a5e: Pushed
4f53326d9ef5: Pushed
810e950c6a17: Pushed
f5192aaff1a0: Pushed
6d0665b308e3: Pushed
1708488e8eef: Pushed
d5202d622907: Pushed
e3589906a2ee: Pushed
e84232dd4559: Pushed
079718509333: Pushed
7292be7bbc48: Pushed
2542fe3ce207: Pushed
6f5c8887c41a: Pushed
001e4a80973b: Pushed
2ba5b91ca2b0: Pushed
2f37d1102187: Pushed
79bde4d54386: Pushed
latest: digest: f27434b1debf742e5a949c7b0951e2a7d3396e785d7f78a443bbe562cf486b56 size: 4718
C:\Users\Basant.Hota>
```

Image will be seen

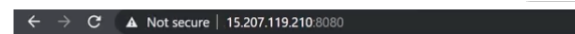
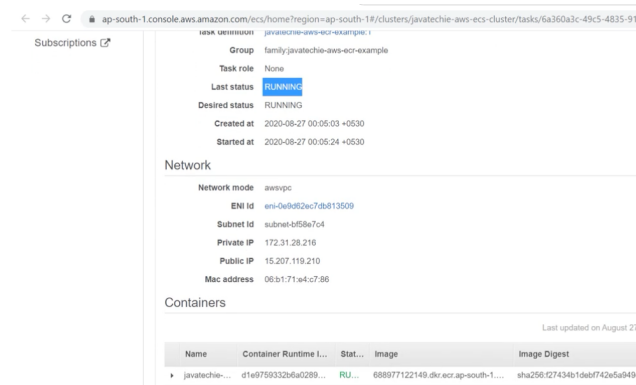
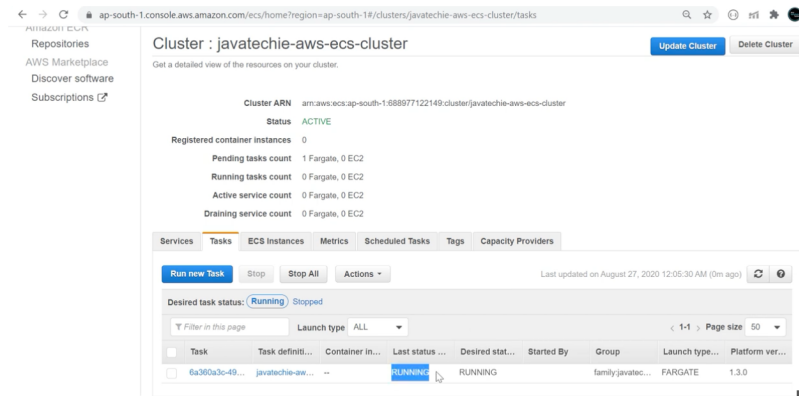


Copy the image URI to add it to task. Refer docker hub steps to create task definition





Now create cluster and add new task as we did in the first half of this document.



Application is up and running !

